세션 기반 보안 파이프라인 — 구성·명령어·코드

2026-11-03 · an sung ju · 9단계 흐름 구현 (실행 확인됨)

1. 구성 (9단계)

#	단계 · 내용 · 코드
1~2	**디자인 패턴 파악 · 점화식 이산화** 웹 프론트의 시간 이산화 값을 격자점으로 시각화해 디자인 패턴을 파악하고, 그 패턴을 점화식으로 이산화한다. 코드: `fit_recurrence, e_ladder, ladder_interval`
3	**과거값 → 쿠키 → WAS** 점화식의 과거 순서에 해당하는 값을 쿠키로 WAS에 넘긴다(서버 서명 HMAC 토큰). 코드: `make_cookie / verify_cookie`
4	**백엔드 도메인 프록시 진입** 그렇게 동작한 백엔드 도메인의 프록시로 진입한다. 쿠키의 과거값이 맞아야 통과한다. 코드: `POST /enter`
5	**현재값 순서 → 세션 ID 생성** request 값에 격자점 점화식의 현재값에 맞는 순서대로 세션을 대입하면 세션 ID가 생성된다. 코드: `make_session_id / POST /session`
6	**미래값 주입 → AES 복호 → response** 미래값에 해당하는 격자점 점화식을 도메인에 주입하면 AES 암호가 풀리면서 response(API)가 나온다. 즉 API의 https/http, LocalC:Local:D. 코드: `seal / unseal / POST /response`
7	**node 접근 → tcp/udp 라우팅** 그 뒤 node를 보기 위해서는 tcp/udp request 라우팅 프로토콜을 거친다. 코드: `GET /node`
8	**DB 점검: query 조건문 확인** 들어온 query에 조건문이 있는지 확인한다. 1차 방어는 매개변수화된 질의, 조건문 탐지는 2차 점검. 코드: `query_has_conditional, safe_query / POST /db`
9	**RSA 스위칭 포인트 체크** 질의를 들여다볼 수 없는 불투명 경로면 RSA 스위칭 포인트를 체크한다. 코드: `RSASwitchPoint`

2. 한 줄 흐름

> 프론트 시간이산화 값 → 격자점 시각화 → 디자인 패턴 → 점화식 이산화 → (과거값=쿠키→WAS) → 백엔드 도메인 프록시 진입 → (현재값 순서로 세션 대입 → 세션 ID 생성) → (미래값 점화식 주입 → AES 복호 → response API: https/http, LocalC:Local:D) → node 접근: tcp/udp 라우팅 → DB 점검: query 조건문 확인 → 안 보이면 RSA 스위칭 포인트 체크

3. 실행 명령어

```
# 0) 의존성
pip install cryptography --break-system-packages

# 1) WAS/백엔드 프록시 (터미널 1) — 서버 비밀은 환경변수로, 디스크에 두지 않음
export SRV_SECRET='change-this-to-32-bytes-secret!!'
PORT=8070 python3 server.py

# 2) 프론트 9단계 실행 (터미널 2)
export SRV_SECRET='change-this-to-32-bytes-secret!!'
BASE=http://127.0.0.1:8070 python3 client.py

# 3) 개별 확인
curl -s http://127.0.0.1:8070/node
curl -s -X POST http://127.0.0.1:8070/db -H 'Content-Type: application/json' -d '{"q":"beta"}'
curl -s -X POST http://127.0.0.1:8070/db -H 'Content-Type: application/json' -d '{"q":"x'\'' OR 1=1 --"}'
```

4. 실행 결과

```
[1~2] 점화식 u_k+1 = 1.000·u_k + 0.900,  사다리 구간=(4.59977,4.60000)
[3] 과거값 쿠키 생성: 00000000000000646fc95864…
[4] 프록시 진입: {'ok': True, 'order': 100}
[5] 세션 ID 생성: f0a9be79b7a2bbd2193298ac…
[6] 미래값 주입 → AES 복호 → response: {'api': 'response-ok', 'routes': {...}}
[7] node 접근(tcp/udp 라우팅): {'routes': {...}, 'transport': 'tcp/udp request routing'}
[8] 정상 질의: {'rows': [[1, 'alpha', 'A-001']]}
[8] 조건문 주입 시도: {'blocked': True, 'reason': 'query에 조건문 감지'}
위조 쿠키 거부: {'error': 'bad cookie (과거값 불일치)'}
```

모든 단계 통과. 조건문 주입과 위조 쿠키는 거부됨. 프로토타입이라 실서비스는 HTTP→HTTPS/TLS, 인메모리 SQLite→실제 DB, LocalC/LocalD·UDP 터널(WireGuard)은 라벨 자리에 실제 주소를 넣는다.

5. 코드 — pipeline.py

pipeline.py

```
"""세션 기반 보안 파이프라인 — 핵심 모듈.

구성(사용자 설계)대로 구현한다.
1. 웹 프론트의 시간 이산화 값을 격자점으로 본다.
2. 패턴을 점화식으로 이산화한다.
3. 점화식의 과거 순서 값 = 쿠키 → WAS.
4. 백엔드 도메인 프록시로 진입.
5. request + 현재값 순서 → 세션 ID 생성.
6. 미래값 점화식 주입 → AES 복호 → response(API).
7. node 접근 → tcp/udp 라우팅.
8. DB 점검: query에 조건문이 있는지 확인.
9. 못 보면 RSA 스위칭 포인트 체크.

점화식·격자는 '중복을 피하는' 용도(값의 표를 저장하지 않고 씨앗 하나로 재생성)이고,
과거·현재·미래 값이 인증에도 쓰이므로 씨앗에는 서버 비밀을 섞는다.
"""
from __future__ import annotations
import hmac, hashlib, math, struct, os
from dataclasses import dataclass
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding


# ─────────────────────────────────────────────────────────── 1~2. 격자와 e 나머지 사다리, 점화식 이산화

def e_ladder(n: int) -> tuple[float, float]:
    """e의 나머지 사다리 구간 [S_n, S_n + 1/(n!·n)].  n이 커질수록 좁아진다."""
    S = sum(1.0 / math.factorial(k) for k in range(n + 1))
    return S, S + 1.0 / (math.factorial(n) * n)


def ladder_interval(value: float, n: int) -> tuple[float, float]:
    """값에 폭 1/(n!·n)짜리 구간을 씌운다.  격자점 값을 넘길 때의 정밀도 조절."""
    w = 1.0 / (math.factorial(n) * n)
    q = math.floor(value / w)
    return q * w, (q + 1) * w


def fit_recurrence(grid_values: list[float]) -> tuple[float, float]:
    """격자점 값 u_k에서 1차 점화식 u_{k+1} = a·u_k + b 를 최소제곱으로 맞춘다.
    프론트의 시간 이산화 값에서 '디자인 패턴'(선형 추세)을 읽는 단계."""
    xs, ys = grid_values[:-1], grid_values[1:]
    nx = len(xs)
    sx, sy = sum(xs), sum(ys)
    sxx = sum(x * x for x in xs)
    sxy = sum(x * y for x, y in zip(xs, ys))
    denom = nx * sxx - sx * sx
    a = (nx * sxy - sx * sy) / denom if denom else 1.0
    b = (sy - a * sx) / nx
    return a, b


# ─────────────────────────────────────────────────────────── 과거·현재·미래 (점화식의 세 순서)

@dataclass
class Triple:
    order: int
    past: bytes      # 순서 order-1
    present: bytes   # 순서 order
    future: bytes    # 순서 order+1
```

```
class Oracle:
    """서버 비밀을 씨앗에 섞은 점화식.  term(k)는 순서 k의 값(16바이트).
    씨앗 하나로 모든 순서를 다시 만든다 → 값의 표를 중복 저장하지 않는다."""

    def __init__(self, server_secret: bytes, context: bytes):
        self.seed = hmac.new(server_secret, b"seed|" + context, hashlib.sha256).digest()

    def term(self, k: int) -> bytes:
        return hmac.new(self.seed, b"term|" + struct.pack(">q", k), hashlib.sha256).digest()[:16]

    def triple(self, order: int) -> Triple:
        return Triple(order, self.term(order - 1), self.term(order), self.term(order + 1))


# ─────────────────────────────────────────────────────────── 3. 과거값 → 쿠키 토큰 → WAS

def make_cookie(server_secret: bytes, past: bytes, order: int) -> str:
    """과거값에 서버 서명(HMAC)을 붙인 쿠키 토큰.  위조 방지."""
    body = struct.pack(">q", order) + past
    sig = hmac.new(server_secret, b"cookie|" + body, hashlib.sha256).digest()[:16]
    return (body + sig).hex()


def verify_cookie(server_secret: bytes, token_hex: str, oracle: Oracle) -> int | None:
    """WAS가 쿠키를 검증.  서명과 과거값이 맞으면 order를 돌려준다."""
    try:
        raw = bytes.fromhex(token_hex)
    except ValueError:
        return None
    body, sig = raw[:-16], raw[-16:]
    if not hmac.compare_digest(sig, hmac.new(server_secret, b"cookie|" + body, hashlib.sha256).digest()[:16]):
        return None
    order = struct.unpack(">q", body[:8])[0]
    past = body[8:]
    return order if hmac.compare_digest(past, oracle.term(order - 1)) else None


# ─────────────────────────────────────────────────────────── 5. 현재값 + request → 세션 ID

def make_session_id(present: bytes, request_value: bytes) -> str:
    """request 값에 현재값 순서를 맞춰 대입 → 세션 ID."""
    return hmac.new(present, b"session|" + request_value, hashlib.sha256).hexdigest()


# ─────────────────────────────────────────────────────────── 6. 미래값 → AES-GCM 키 → response 복호

def future_key(future: bytes, session_id: str) -> bytes:
    """미래값과 세션으로 AES-256 키를 유도(HKDF)."""
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=session_id.encode(),
                info=b"aes-response").derive(future)


def seal(future: bytes, session_id: str, plaintext: bytes) -> str:
    key = future_key(future, session_id)
    nonce = os.urandom(12)
    ct = AESGCM(key).encrypt(nonce, plaintext, session_id.encode())
    return (nonce + ct).hex()


def unseal(future: bytes, session_id: str, blob_hex: str) -> bytes:
    """미래값 점화식을 주입하면 AES 암호가 풀린다."""
    key = future_key(future, session_id)
    raw = bytes.fromhex(blob_hex)
    return AESGCM(key).decrypt(raw[:12], raw[12:], session_id.encode())


# ─────────────────────────────────────────────────────────── 8. DB 점검: query에 조건문이 있는지 확인

_COND_TOKENS = (" or ", " and ", "--", ";", "/*", " union ", "' or ", "=1", "1=1")
```

```
def query_has_conditional(raw_input: str) -> bool:
    """입력 문자열에 조건문/주입 흔적이 있는지 본다(2차 점검)."""
    s = " " + raw_input.lower().replace("'", "' ") + " "
    return any(tok in s for tok in _COND_TOKENS)


def safe_query(conn, sql_template: str, params: tuple):
    """1차 방어는 매개변수화된 질의.  값과 명령을 분리해 조건문 주입을 원천 차단."""
    cur = conn.cursor()
    cur.execute(sql_template, params)   # 템플릿은 상수, params는 ?로만 들어간다
    return cur.fetchall()


# ──────────────────────────────────────────────────────────────────── 9. 못 보면 RSA 스위칭 포인트 체크

class RSASwitchPoint:
    """질의를 들여다볼 수 없을 때(불투명 경로) 비대칭 서명으로 경로를 재확인한다."""

    def __init__(self):
        self.key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.pub = self.key.public_key()

    def sign(self, challenge: bytes) -> bytes:
        return self.key.sign(challenge, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                             salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())

    def verify(self, challenge: bytes, sig: bytes) -> bool:
        try:
            self.pub.verify(sig, challenge, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                            salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
            return True
        except Exception:
            return False
```

6. 코드 — server.py

server.py

```
"""WAS/백엔드 도메인 프록시 — http.server.

엔드포인트(= response api의 https/http):
  POST /enter    : 4. 프록시 진입 — 쿠키(과거값) 검증 → 세션 발급 예약
  POST /session  : 5. request + 현재값 순서 → 세션 ID 생성
  POST /response : 6. 미래값 주입 → AES 복호 → 암호화된 response 반환
  GET  /node     : 7. node 접근 — tcp/udp 라우팅 뒤에서만 보임
  POST /db       : 8~9. DB 점검(query 조건문 확인), 못 보면 RSA 스위칭 포인트 체크

로컬 라우팅 라벨(LocalC / LocalD)은 설정으로 둔다 — 실제 로컬 서비스 주소를 넣는 자리.
"""
import json, sqlite3, os, struct
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
import pipeline as P

SERVER_SECRET = os.environ.get("SRV_SECRET", "demo-server-secret-32byte-string!").encode()[:32].ljust(32, b"0")
ROUTES = {"LocalC": "127.0.0.1:7001", "LocalD": "127.0.0.1:7002"}   # LocalC:Local:D 자리
RSA_SWITCH = P.RSASwitchPoint()

# 데모용 인메모리 DB
_db = sqlite3.connect(":memory:", check_same_thread=False)
_db.execute("CREATE TABLE node(id INTEGER PRIMARY KEY, name TEXT, value TEXT)")
_db.executemany("INSERT INTO node(name, value) VALUES(?,?)",
                [("alpha", "A-001"), ("beta", "B-002"), ("gamma", "C-003")])
_db.commit()

SESSIONS = {}   # session_id -> {"order":k, "context":...}


def oracle_for(context: bytes) -> P.Oracle:
    return P.Oracle(SERVER_SECRET, context)


class Handler(BaseHTTPRequestHandler):
    def _body(self):
        n = int(self.headers.get("Content-Length", 0))
        return json.loads(self.rfile.read(n) or b"{}")

    def _send(self, code, obj):
        data = json.dumps(obj).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

    def log_message(self, *a):  # 조용히
        pass

    def do_GET(self):
        if self.path.startswith("/node"):
            # 7. node 접근: tcp/udp 라우팅을 통과한 요청만 라우팅표를 받는다
            self._send(200, {"routes": ROUTES, "transport": "tcp/udp request routing"})
        else:
            self._send(404, {"error": "not found"})

    def do_POST(self):
        try:
            b = self._body()
            if self.path == "/enter":          # 4. 프록시 진입
                ctx = b["context"].encode()
                order = P.verify_cookie(SERVER_SECRET, b["cookie"], oracle_for(ctx))
                if order is None:
                    return self._send(401, {"error": "bad cookie (과거값 불일치)"})
```

```
                return self._send(200, {"ok": True, "order": order})

            if self.path == "/session":        # 5. 세션 ID 생성
                ctx = b["context"].encode()
                order = b["order"]
                present = oracle_for(ctx).term(order)
                sid = P.make_session_id(present, b["request"].encode())
                SESSIONS[sid] = {"order": order, "context": b["context"]}
                return self._send(200, {"session_id": sid})

            if self.path == "/response":       # 6. 미래값 → AES 복호 → response
                sid = b["session_id"]
                if sid not in SESSIONS:
                    return self._send(401, {"error": "no session"})
                s = SESSIONS[sid]
                future = oracle_for(s["context"].encode()).term(s["order"] + 1)
                # 서버가 응답을 봉인(미래값 키로). 클라이언트가 같은 미래값으로 풀 수 있다.
                payload = json.dumps({"api": "response-ok", "routes": ROUTES}).encode()
                blob = P.seal(future, sid, payload)
                return self._send(200, {"sealed": blob})

            if self.path == "/db":             # 8~9. DB 점검
                raw = b.get("q", "")
                if P.query_has_conditional(raw):
                    return self._send(400, {"blocked": True, "reason": "query에 조건문 감지"})
                try:
                    rows = P.safe_query(_db, "SELECT id,name,value FROM node WHERE name=?", (raw,))
                    return self._send(200, {"rows": rows})
                except sqlite3.Error:
                    # 질의를 들여다볼 수 없는 불투명 경로 → RSA 스위칭 포인트 체크
                    chal = os.urandom(16)
                    sig = RSA_SWITCH.sign(chal)
                    ok = RSA_SWITCH.verify(chal, sig)
                    return self._send(200, {"rsa_switch_checked": ok, "challenge": chal.hex()})

            self._send(404, {"error": "not found"})
        except Exception as e:
            self._send(500, {"error": str(e)})


if __name__ == "__main__":
    port = int(os.environ.get("PORT", "8070"))
    srv = ThreadingHTTPServer(("127.0.0.1", port), Handler)
    print(f"WAS/proxy on http://127.0.0.1:{port}  routes={ROUTES}", flush=True)
    srv.serve_forever()
```

7. 코드 — client.py

client.py

```
"""프론트 — 9단계 흐름을 순서대로 실행한다.

쿠키(과거값)는 세션 발급 주체인 서버가 서명해 주므로, 데모에서는 서버와 같은
비밀로 oracle을 만들어 과거값을 얻는다(실제로는 로그인 응답으로 내려받는다).
"""
import json, os, urllib.request
import pipeline as P

BASE = os.environ.get("BASE", "http://127.0.0.1:8070")
SERVER_SECRET = os.environ.get("SRV_SECRET", "demo-server-secret-32byte-string!").encode()[:32].ljust(32, b"0")
CONTEXT = "user=alice|device=42"
ORDER = 100


def post(path, obj):
    req = urllib.request.Request(BASE + path, data=json.dumps(obj).encode(),
                                 headers={"Content-Type": "application/json"}, method="POST")
    try:
        with urllib.request.urlopen(req) as r:
            return json.load(r)
    except urllib.error.HTTPError as e:     # 차단(400/401)도 정상 결과로 읽는다
        return json.load(e)


def get(path):
    with urllib.request.urlopen(BASE + path) as r:
        return json.load(r)


def main():
    # 1~2. 프론트의 시간 이산화 값을 격자점으로 보고 점화식(패턴)으로 이산화
    grid = [1.0, 1.9, 2.8, 3.7, 4.6]                # 시간 이산화 값(격자점)
    a, b = P.fit_recurrence(grid)
    lo, hi = P.ladder_interval(grid[-1], n=6)         # e 나머지 사다리로 구간 선택
    print(f"[1~2] 점화식 u_k+1 = {a:.3f}·u_k + {b:.3f},  사다리 구간=({lo:.5f},{hi:.5f})")

    # 점화식의 세 순서(과거·현재·미래). 씨앗 하나로 재생성 → 중복 저장 없음
    orc = P.Oracle(SERVER_SECRET, CONTEXT.encode())
    tri = orc.triple(ORDER)

    # 3. 과거값 → 쿠키 → WAS
    cookie = P.make_cookie(SERVER_SECRET, tri.past, ORDER)
    print(f"[3] 과거값 쿠키 생성: {cookie[:24]}…")

    # 4. 백엔드 도메인 프록시 진입
    r = post("/enter", {"context": CONTEXT, "cookie": cookie})
    print(f"[4] 프록시 진입: {r}")

    # 5. request + 현재값 순서 → 세션 ID
    r = post("/session", {"context": CONTEXT, "order": ORDER, "request": "GET /node"})
    sid = r["session_id"]
    print(f"[5] 세션 ID 생성: {sid[:24]}…")

    # 6. 미래값 주입 → AES 복호 → response(API)
    r = post("/response", {"session_id": sid})
    future = orc.term(ORDER + 1)
    plain = P.unseal(future, sid, r["sealed"])
    print(f"[6] 미래값 주입 → AES 복호 → response: {json.loads(plain)}")

    # 7. node 접근 — tcp/udp 라우팅
    r = get("/node")
    print(f"[7] node 접근(tcp/udp 라우팅): {r}")

    # 8. DB 점검 — 정상 질의 / 조건문 주입 시도
```

```
    ok = post('/db', {'q': 'alpha'})
    inj = post('/db', {'q': "x' OR 1=1 --"})
    print(f"[8] 정상 질의: {ok}")
    print(f"[8] 조건문 주입 시도: {inj}")


if __name__ == "__main__":
    main()
```